

Name: Monisha Raghini Chelle

RollNo:2520030537

Section-6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

15.To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSSP-Practical      foldername  moni    sample1.c
OSLAB                          Simple-Commands-Week1 hello       monis   sample1.c.save
OSLAB14                        example.txt         hello.c     sample1
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello  hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c  hello:Zone.Identifier    hello_shortcut skill
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2 — Create the Combined Redirection Program

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano combined_redirection.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 combined_redirection.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define INPUT_SIZE 200

int main()
{
    char input[INPUT_SIZE];
    int saved_stdout;
    int saved_stderr;
    int fd;

    printf("Combined Redirection Program\n");
    printf("Enter: test > output.txt 2>&1\n");
    printf("Type exit to quit.\n");

    while (1)
    {
        printf("combined> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        char *redirect = strstr(input, "2>&1");

        if (redirect == NULL)
        {
            printf("Error: Combined redirection '2>&1' not found.\n");
            continue;
        }

        *redirect = '\0';

        char *output_redirect = strchr(input, '>');

        if (output_redirect == NULL)
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 combined_redirection.c *
if (output_redirect == NULL)
{
    printf("Error: Output redirection '>' not found.\n");
    continue;
}

*output_redirect = '\0';

char *command = input;
char *filename = output_redirect + 1;

while (*command == ' ')
    command++;

while (*filename == ' ')
    filename++;

if (*command == '\0')
{
    printf("Error: Command is missing.\n");
    continue;
}

if (*filename == '\0')
{
    printf("Error: Output file name is missing.\n");
    continue;
}

fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (fd == -1)
{
    perror("open");
    printf("Error: Cannot open output file: %s\n", filename);
    continue;
}

saved_stdout = dup(STDOUT_FILENO);
saved_stderr = dup(STDERR_FILENO);

if (saved_stdout == -1 || saved_stderr == -1)
{
    perror("dup");
    close(fd);
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line  M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 combined_redirection.c *

    continue;
}

if (dup2(fd, STDOUT_FILENO) == -1)
{
    perror("dup2 stdout");
    close(fd);
    close(saved_stdout);
    close(saved_stderr);
    continue;
}

/*
 * stderr is duplicated from stdout.
 * Therefore stdout and stderr now use the same file.
 */
if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1)
{
    perror("dup2 stderr");
    close(fd);
    close(saved_stdout);
    close(saved_stderr);
    continue;
}

close(fd);

printf("Command output: %s\n", command);
fprintf(stderr, "Error output: simulated error message\n");

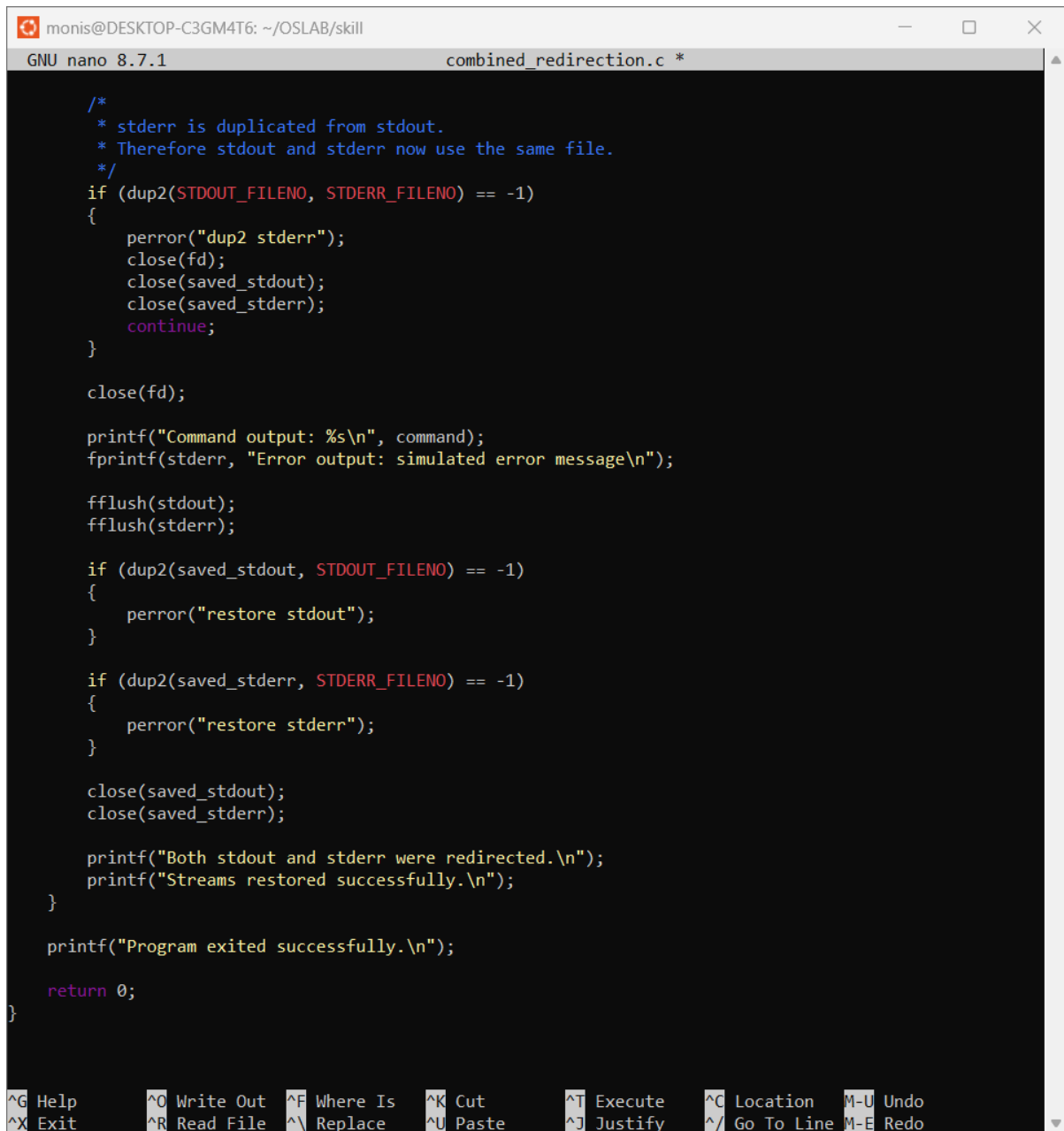
fflush(stdout);
fflush(stderr);

if (dup2(saved_stdout, STDOUT_FILENO) == -1)
{
    perror("restore stdout");
}

if (dup2(saved_stderr, STDERR_FILENO) == -1)
{
    perror("restore stderr");
}

close(saved_stdout);
close(saved_stderr);

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 combined_redirection.c *

/*
 * stderr is duplicated from stdout.
 * Therefore stdout and stderr now use the same file.
 */
if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1)
{
    perror("dup2 stderr");
    close(fd);
    close(saved_stdout);
    close(saved_stderr);
    continue;
}

close(fd);

printf("Command output: %s\n", command);
fprintf(stderr, "Error output: simulated error message\n");

fflush(stdout);
fflush(stderr);

if (dup2(saved_stdout, STDOUT_FILENO) == -1)
{
    perror("restore stdout");
}

if (dup2(saved_stderr, STDERR_FILENO) == -1)
{
    perror("restore stderr");
}

close(saved_stdout);
close(saved_stderr);

printf("Both stdout and stderr were redirected.\n");
printf("Streams restored successfully.\n");
}

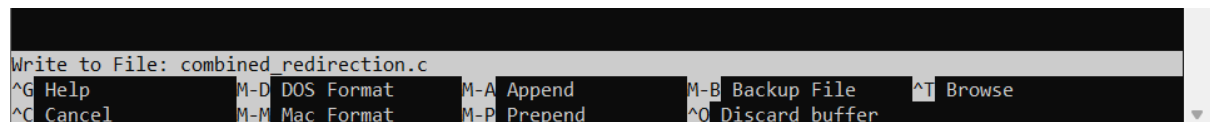
printf("Program exited successfully.\n");

return 0;
}

^G Help      ^O Write Out  ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File  ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

Save:

1. **Ctrl + O**
2. Press **Enter**
3. **Ctrl + X**



Compile:

```
gcc combined_redirection.c -o combined_redirection
```

Verify:

```
ls -l combined_redirection.c combined_redirection
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano combined_redirection.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc combined_redirection.c -o combined_redirection
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l combined_redirection.c combined_redirection
-rwxr-xr-x 1 monis monis 16696 Sep 18 14:07 combined_redirection
-rw-r--r-- 1 monis monis 3190 Sep 18 14:06 combined_redirection.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 3 — Test Combined Redirection

Run:

`./combined_redirection`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./combined_redirection
Combined Redirection Program
Enter: test > output.txt 2>&1
Type exit to quit.
combined>
```

Now enter:

`test > combined.txt 2>&1`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./combined_redirection
Combined Redirection Program
Enter: test > output.txt 2>&1
Type exit to quit.
combined> test > combined.txt 2>&1
Both stdout and stderr were redirected.
Streams restored successfully.
combined>
```

Now exit:

`exit`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./combined_redirection
Combined Redirection Program
Enter: test > output.txt 2>&1
Type exit to quit.
combined> test > combined.txt 2>&1
Both stdout and stderr were redirected.
Streams restored successfully.
combined> exit
Program exited successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Verify the combined output

Run:

cat combined.txt

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat combined.txt
cat: combined.txt: No such file or directory
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls
append.txt          escape_parser       memory3.c           process
append_redirection  escape_parser.c    memory3.c.save     process.c
append_redirection.c execution_error     memory4             process_pipeline
builtin_dispatch     execution_error.c  memory4.c          process_pipeline.c
builtin_dispatch.c   export_child       multiple_pipes      shell
builtins            export_child.c    multiple_pipes.c   shell.c
builtins.c          export_variables   output.txt          single_pipe
child_process       export_variables.c output_redirection  single_pipe.c
child_process.c     history           output_redirection.c single_quotes
'combined.txt '     history.c         parent_child        single_quotes.c
combined_redirection input.txt         parent_child.c      stderr_redirection
combined_redirection.c input_redirection parser_logic        stderr_redirection.c
command_history     input_redirection.c parser_logic.c      tokenization
command_history.c   long_pipeline     pass_arguments      tokenization.c
command_launch      long_pipeline.c   pass_arguments.c    tokenization.c.save
command_launch.c    memory            path_missing        tokenization.c
directory_navigation memory.4          path_missing.c      variable_expansion
directory_navigation.c memory.c          path_resolution     variable_expansion.c
directory_state     memory1           path_resolution.c   waitpid_monitor
directory_state.c   memory2           pipe_sync            waitpid_monitor.c
double_quotes       memory2.c        pipeline_structure  waitpid_monitor.c
double_quotes.c     memory3           pipeline_structure.c
error.txt           memory3.c        pipeline_structure.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ mv "combined.txt" "combined.txt"
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat combined.txt
Command output: test
Error output: simulated error message
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 4 — Test Combined Redirection Edge Cases.

1. Open your Ubuntu terminal

Run:

cd ~/OSLAB/skill

./combined_redirection

2. Test a missing output filename

At the combined> prompt, enter:

test > 2>&1

3. Test a missing command

Enter:

```
> edge_case.txt 2>&1
```

4. Test a missing combined-redirection operator

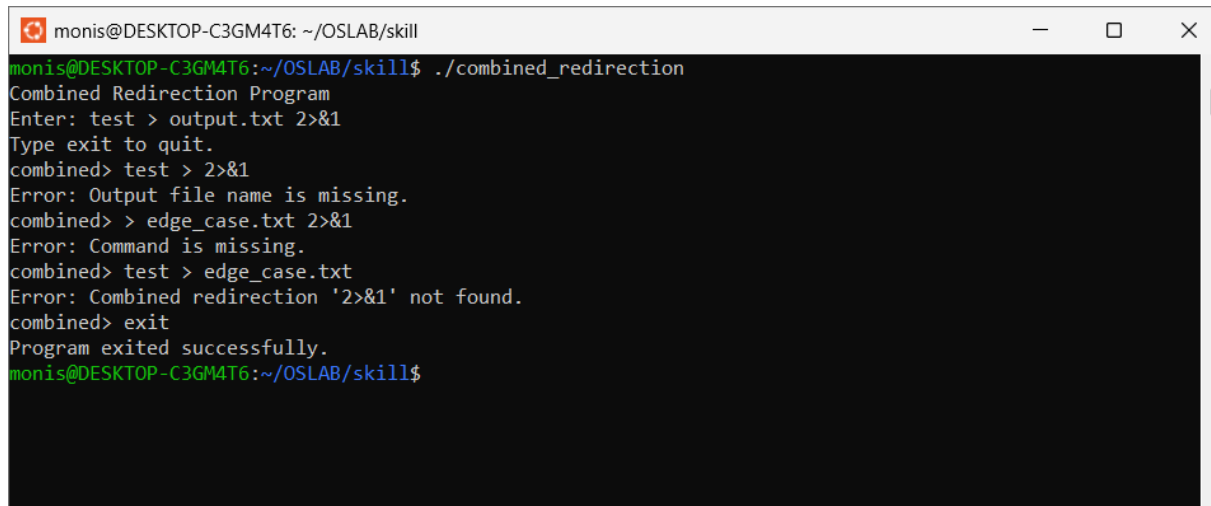
Enter:

```
test > edge_case.txt
```

5. Exit the program

Enter:

```
exit
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./combined_redirection
Combined Redirection Program
Enter: test > output.txt 2>&1
Type exit to quit.
combined> test > 2>&1
Error: Output file name is missing.
combined> > edge_case.txt 2>&1
Error: Command is missing.
combined> test > edge_case.txt
Error: Combined redirection '2>&1' not found.
combined> exit
Program exited successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

2

Step 5: Create the Complex Commands Program

1. Open Ubuntu and create the file

```
cd ~/OSLAB/skill
```

```
nano complex_commands.c
```

2. Complete C code

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 complex_commands.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define SIZE 300

/* Remove leading and trailing spaces */
char *trim(char *str)
{
    while (isspace((unsigned char)*str))
        str++;

    if (*str == '\0')
        return str;

    char *end = str + strlen(str) - 1;

    while (end > str && isspace((unsigned char)*end))
    {
        *end = '\0';
        end--;
    }

    return str;
}

int main()
{
    char input[SIZE];
    char *input_file;
    char *output_file;
    char *search_word;

    int pipefd[2];
    pid_t pid1, pid2;

    printf("Complex Commands Program\n");
    printf("Supported format:\n");
    printf("cat < input.txt | grep WORD > output.txt\n");

    printf("\nEnter command: ");

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
printf("\nEnter command: ");
fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
    return 1;

input[strcspn(input, "\n")] = '\0';

/* Locate the operators */
char *in_op = strchr(input, '<');
char *pipe_op = strchr(input, '|');
char *out_op = strchr(input, '>');

/* Validate operator presence and order */
if (in_op == NULL || pipe_op == NULL || out_op == NULL ||
    !(in_op < pipe_op && pipe_op < out_op))
{
    printf("Error: Invalid command syntax.\n");
    printf("Expected: cat < input.txt | grep WORD > output.txt\n");
    return 1;
}

/* Split command into sections */
*in_op = '\0';
*pipe_op = '\0';
*out_op = '\0';

char *first_command = trim(input);
input_file = trim(in_op + 1);

char *second_section = trim(pipe_op + 1);
output_file = trim(out_op + 1);

/* Validate the first command */
if (strcmp(first_command, "cat") != 0)
{
    printf("Error: First command must be cat.\n");
    return 1;
}

/* Validate the second command */
if (strncmp(second_section, "grep ", 5) != 0)
{
    printf("Error: Second command must be grep WORD.\n");
    return 1;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

```
}

search_word = trim(second_section + 5);

/* Validate filenames and search word */
if (*input_file == '\0' ||
    *output_file == '\0' ||
    *search_word == '\0')
{
    printf("Error: Missing filename or search word.\n");
    return 1;
}

/* Reject unsupported extra operators */
if (strpbrk(input_file, "<>|") != NULL ||
    strpbrk(output_file, "<>|") != NULL ||
    strpbrk(search_word, "<>|") != NULL)
{
    printf("Error: Unsupported extra redirection or pipe.\n");
    return 1;
}

/* Check the input file before creating the output file */
int input_fd = open(input_file, O_RDONLY);

if (input_fd == -1)
{
    perror("Input file");
    return 1;
}

/* Create the output file */
int output_fd = open(output_file,
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

if (output_fd == -1)
{
    perror("Output file");
    close(input_fd);
    return 1;
}

/* Display the execution plan */
printf("\nExecution Plan:\n");
```

```
/* Display the execution plan */
printf("\nExecution Plan:\n");
printf("1. Open input file: %s\n", input_file);
printf("2. Execute cat\n");
printf("3. Send cat output through a pipe\n");
printf("4. Execute grep with search word: %s\n", search_word);
printf("5. Redirect grep output to: %s\n", output_file);

fflush(stdout);

/* Create a pipe */
if (pipe(pipefd) == -1)
{
    perror("pipe");
    close(input_fd);
    close(output_fd);
    return 1;
}

/* First child: cat */
pid1 = fork();

if (pid1 < 0)
{
    perror("fork");
    close(input_fd);
    close(output_fd);
    close(pipefd[0]);
    close(pipefd[1]);
    return 1;
}

if (pid1 == 0)
{
    /* Redirect input file to stdin */
    if (dup2(input_fd, STDIN_FILENO) == -1)
    {
        perror("dup2 input");
        _exit(1);
    }

    /* Redirect stdout to the pipe */
    if (dup2(pipefd[1], STDOUT_FILENO) == -1)
    {
        perror("dup2 pipe");
    }
}
```

```
        perror("dup2 pipe");
        _exit(1);
    }

    close(input_fd);
    close(output_fd);
    close(pipefd[0]);
    close(pipefd[1]);

    execlp("cat", "cat", (char *)NULL);

    perror("cat");
    _exit(1);
}

/* Second child: grep */
pid2 = fork();

if (pid2 < 0)
{
    perror("fork");

    close(input_fd);
    close(output_fd);
    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    return 1;
}

if (pid2 == 0)
{
    /* Read input from the pipe */
    if (dup2(pipefd[0], STDIN_FILENO) == -1)
    {
        perror("dup2 pipe input");
        _exit(1);
    }

    /* Redirect output to the output file */
    if (dup2(output_fd, STDOUT_FILENO) == -1)
    {
        perror("dup2 output");
        _exit(1);
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

monis@DESKTOP-C3GM4T6: ~/OSLAB/skill

GNU nano 8.7.1

complex_commands.c *

```
        perror("dup2 output");
        _exit(1);
    }

    close(input_fd);
    close(output_fd);
    close(pipefd[0]);
    close(pipefd[1]);

    execlp("grep", "grep", search_word, (char *)NULL);

    perror("grep");
    _exit(1);
}

/* Parent closes unused descriptors */
close(input_fd);
close(output_fd);
close(pipefd[0]);
close(pipefd[1]);

/* Wait for both children */
int status1, status2;

waitpid(pid1, &status1, 0);
waitpid(pid2, &status2, 0);

if (!WIFEXITED(status1) || WEXITSTATUS(status1) != 0)
{
    printf("Error: cat failed.\n");
    return 1;
}

if (!WIFEXITED(status2))
{
    printf("Error: grep failed.\n");
    return 1;
}

if (WEXITSTATUS(status2) == 1)
{
    printf("Pipeline completed. No matching lines found.\n");
    return 0;
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 complex_commands.c *
    _exit(1);
}

/* Parent closes unused descriptors */
close(input_fd);
close(output_fd);
close(pipefd[0]);
close(pipefd[1]);

/* Wait for both children */
int status1, status2;

waitpid(pid1, &status1, 0);
waitpid(pid2, &status2, 0);

if (!WIFEXITED(status1) || WEXITSTATUS(status1) != 0)
{
    printf("Error: cat failed.\n");
    return 1;
}

if (!WIFEXITED(status2))
{
    printf("Error: grep failed.\n");
    return 1;
}

if (WEXITSTATUS(status2) == 1)
{
    printf("Pipeline completed. No matching lines found.\n");
    return 0;
}

if (WEXITSTATUS(status2) != 0)
{
    printf("Error: grep failed.\n");
    return 1;
}

printf("\nPipeline executed successfully.\n");
printf("Output saved to: %s\n", output_file);

return 0;
}

Write to File: complex_commands.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

3. Save the file

In nano, press **Ctrl + O**, **Enter**, then **Ctrl + X**.

```
Write to File: complex_commands.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

4. Compile the program

Run:

```
gcc -Wall -Wextra complex_commands.c -o complex_commands
```

If compilation succeeds, no error messages should appear.

Verify both files:

```
ls -l complex_commands.c complex_commands
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano complex_commands.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra complex_commands.c -o complex_commands
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l complex_commands.c complex_commands
-rwxr-xr-x 1 monis monis 16944 Sep 25 11:45 complex_commands
-rw-r--r-- 1 monis monis  6003 Sep 25 11:45 complex_commands.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 6: Test Pipes and Redirections

```
GNU nano 8.7.1 complex_commands1.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define SIZE 300

/* Remove leading and trailing whitespace */
char *trim(char *str)
{
    while (isspace((unsigned char)*str))
        str++;

    if (*str == '\0')
        return str;

    char *end = str + strlen(str) - 1;

    while (end > str && isspace((unsigned char)*end))
    {
        *end = '\0';
        end--;
    }

    return str;
}

int main()
{
    char input[SIZE];

    printf("Complex Commands Program\n");
    printf("Supported command:\n");
    printf("cat < input.txt | grep WORD > output.txt\n\n");
    printf("Enter command: ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        printf("Error: Unable to read command.\n");
        return 1;
    }

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
input[strcspn(input, "\n")] = '\0';

/* Identify redirection and pipe operators */
char *in_op = strchr(input, '<');
char *pipe_op = strchr(input, '|');
char *out_op = strchr(input, '>');

/* Validate the required operators and their order */
if (in_op == NULL ||
    pipe_op == NULL ||
    out_op == NULL ||
    !(in_op < pipe_op && pipe_op < out_op))
{
    printf("Error: Invalid command syntax.\n");
    printf("Expected: cat < input.txt | grep WORD > output.txt\n");
    return 1;
}

/* Separate the command into sections */
*in_op = '\0';
*pipe_op = '\0';
*out_op = '\0';

char *first_command = trim(input);
char *input_file = trim(in_op + 1);
char *second_command = trim(pipe_op + 1);
char *output_file = trim(out_op + 1);

/* Validate the first command */
if (strcmp(first_command, "cat") != 0)
{
    printf("Error: First command must be cat.\n");
    return 1;
}

/* Validate the second command */
if (strncmp(second_command, "grep ", 5) != 0)
{
    printf("Error: Second command must be grep WORD.\n");
    return 1;
}

char *search_word = trim(second_command + 5);

/* Check for missing filenames or search words */
```

```
input[strcspn(input, "\n")] = '\0';

/* Identify redirection and pipe operators */
char *in_op = strchr(input, '<');
char *pipe_op = strchr(input, '|');
char *out_op = strchr(input, '>');

/* Validate the required operators and their order */
if (in_op == NULL ||
    pipe_op == NULL ||
    out_op == NULL ||
    !(in_op < pipe_op && pipe_op < out_op))
{
    printf("Error: Invalid command syntax.\n");
    printf("Expected: cat < input.txt | grep WORD > output.txt\n");
    return 1;
}

/* Separate the command into sections */
*in_op = '\0';
*pipe_op = '\0';
*out_op = '\0';

char *first_command = trim(input);
char *input_file = trim(in_op + 1);
char *second_command = trim(pipe_op + 1);
char *output_file = trim(out_op + 1);

/* Validate the first command */
if (strcmp(first_command, "cat") != 0)
{
    printf("Error: First command must be cat.\n");
    return 1;
}

/* Validate the second command */
if (strncmp(second_command, "grep ", 5) != 0)
{
    printf("Error: Second command must be grep WORD.\n");
    return 1;
}

char *search_word = trim(second_command + 5);

/* Check for missing filenames or search words */
```

monis@DESKTOP-C3GM4T6: ~/OSLAB/skill

GNU nano 8.7.1

complex_commands1.c *

```
/* Check for missing filenames or search words */
if (*input_file == '\0' ||
    *output_file == '\0' ||
    *search_word == '\0')
{
    printf("Error: Missing filename or search word.\n");
    return 1;
}

/* Reject unsupported extra operators */
if (strpbrk(input_file, "<>|") != NULL ||
    strpbrk(output_file, "<>|") != NULL ||
    strpbrk(search_word, "<>|") != NULL)
{
    printf("Error: Unsupported extra redirection or pipe.\n");
    return 1;
}

/* Open the input file */
int input_fd = open(input_file, O_RDONLY);

if (input_fd == -1)
{
    perror("Input file");
    return 1;
}

/* Open or create the output file */
int output_fd = open(
    output_file,
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);

if (output_fd == -1)
{
    perror("Output file");
    close(input_fd);
    return 1;
}

/* Display the execution plan */
printf("\nExecution Plan:\n");
printf("1. Open input file: %s\n", input_file);
printf("2. Execute cat\n");
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^/ Go To Line	M-E Redo

```
printf("2. Execute cat\n");
printf("3. Send cat output through a pipe\n");
printf("4. Execute grep with search word: %s\n", search_word);
printf("5. Redirect grep output to: %s\n", output_file);
```

```
fflush(stdout);
```

```
/* Create the pipe */
```

```
int pipefd[2];
```

```
if (pipe(pipefd) == -1)
{
```

```
    perror("pipe");
```

```
    close(input_fd);
    close(output_fd);
```

```
    return 1;
```

```
}
```

```
/* Create the first child process */
```

```
pid_t pid1 = fork();
```

```
if (pid1 < 0)
```

```
{
```

```
    perror("fork");
```

```
    close(input_fd);
    close(output_fd);
    close(pipefd[0]);
    close(pipefd[1]);
```

```
    return 1;
```

```
}
```

```
if (pid1 == 0)
```

```
{
```

```
    /* Redirect stdin from the input file */
```

```
    if (dup2(input_fd, STDIN_FILENO) == -1)
```

```
    {
```

```
        perror("dup2 input");
        _exit(1);
```

```
    }
```

```
    /* Redirect stdout to the pipe */
```

```
/* Redirect stdout to the pipe */
if (dup2(pipefd[1], STDOUT_FILENO) == -1)
{
    perror("dup2 pipe output");
    _exit(1);
}

close(input_fd);
close(output_fd);
close(pipefd[0]);
close(pipefd[1]);

/* Execute cat */
execlp("cat", "cat", (char *)NULL);

perror("cat");
_exit(1);
}

/* Create the second child process */
pid_t pid2 = fork();

if (pid2 < 0)
{
    perror("fork");

    close(input_fd);
    close(output_fd);
    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);

    return 1;
}

if (pid2 == 0)
{
    /* Redirect stdin from the pipe */
    if (dup2(pipefd[0], STDIN_FILENO) == -1)
    {
        perror("dup2 pipe input");
        _exit(1);
    }
}
```

```
}

/* Redirect stdout to the output file */
if (dup2(output_fd, STDOUT_FILENO) == -1)
{
    perror("dup2 output file");
    _exit(1);
}

close(input_fd);
close(output_fd);
close(pipefd[0]);
close(pipefd[1]);

/* Execute grep */
execlp(
    "grep",
    "grep",
    search_word,
    (char *)NULL
);

perror("grep");
_exit(1);
}

/* Parent closes all file descriptors */
close(input_fd);
close(output_fd);
close(pipefd[0]);
close(pipefd[1]);

/* Wait for both child processes */
int status1;
int status2;

if (waitpid(pid1, &status1, 0) == -1)
{
    perror("waitpid cat");
    return 1;
}

if (waitpid(pid2, &status2, 0) == -1)
{
    perror("waitpid grep");
}
```

```
if (waitpid(pid1, &status1, 0) == -1)
{
    perror("waitpid cat");
    return 1;
}

if (waitpid(pid2, &status2, 0) == -1)
{
    perror("waitpid grep");
    return 1;
}

/* Verify execution results */
if (!WIFEXITED(status1) ||
    WEXITSTATUS(status1) != 0)
{
    printf("Error: cat failed.\n");
    return 1;
}

if (!WIFEXITED(status2))
{
    printf("Error: grep failed.\n");
    return 1;
}

if (WEXITSTATUS(status2) == 1)
{
    printf("\nPipeline completed.\n");
    printf("No matching lines found.\n");
    return 0;
}

if (WEXITSTATUS(status2) != 0)
{
    printf("Error: grep failed.\n");
    return 1;
}

printf("\nPipeline executed successfully.\n");
printf("Output saved to: %s\n", output_file);

return 0;
}
```

^G Help

^O Write Out

^F Where Is

^K Cut

^T Execute

^C Location

M-U Undo

^X Exit

^R Read File

^_ Replace

^U Paste

^J Justify

^_ Go To Line

M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 complex_commands1.c *
if (waitpid(pid1, &status1, 0) == -1)
{
    perror("waitpid cat");
    return 1;
}

if (waitpid(pid2, &status2, 0) == -1)
{
    perror("waitpid grep");
    return 1;
}

/* Verify execution results */
if (!WIFEXITED(status1) ||
    WEXITSTATUS(status1) != 0)
{
    printf("Error: cat failed.\n");
    return 1;
}

if (!WIFEXITED(status2))
{
    printf("Error: grep failed.\n");
    return 1;
}

if (WEXITSTATUS(status2) == 1)
{
    printf("\nPipeline completed.\n");
    printf("No matching lines found.\n");
    return 0;
}

if (WEXITSTATUS(status2) != 0)
{
    printf("Error: grep failed.\n");
    return 1;
}

printf("\nPipeline executed successfully.\n");
printf("Output saved to: %s\n", output_file);

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

3. Save and compile

Press Ctrl + O, Enter, then Ctrl + X.

```
Write to File: complex_commands1.c
^G Help   M-D DOS Format   M-A Append   M-B Backup File   ^T Browse
^C Cancel   M-M Mac Format   M-P Prepend   ^O Discard buffer
```

Compile:

gcc -Wall -Wextra complex_commands.c -o complex_commands

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano complex_commands1.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra complex_commands.c -o complex_commands
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

4. Create the test input file

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano input1.txt
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 input1.txt *
OSSP Practical Q15
Operating Systems Laboratory
Linux Programming
OS Process Management
Computer Networks
```

5.Run the program

`./complex_commands`

At Enter command:, type:

`cat < input1.txt | grep OS > output1.txt`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./complex_commands
Complex Commands Program
Supported format:
cat < input.txt | grep WORD > output.txt

Enter command: cat < input1.txt | grep OS > output1.txt

Execution Plan:
1. Open input file: input1.txt
2. Execute cat
3. Send cat output through a pipe
4. Execute grep with search word: OS
5. Redirect grep output to: output1.txt

Pipeline executed successfully.
Output saved to: output1.txt
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

6. Verify the result

Verify:

cat output1.txt

wc -l output1.txt

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat output1.txt
OSSP Practical Q15
OS Process Management
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ wc -l output1.txt
2 output1.txt
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 7 — Test invalid syntax and missing files

Test A — Missing pipe

`./complex_commands`

Enter:

`cat < input1.txt grep OS > output1.txt`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./complex_commands
Complex Commands Program
Supported format:
cat < input.txt | grep WORD > output.txt

Enter command: cat < input1.txt grep OS > output1.txt
Error: Invalid command syntax.
Expected: cat < input.txt | grep WORD > output.txt
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Test B — Missing input file

Run:

`./complex_commands`

Enter:

`cat < missing_q15.txt | grep OS > output1.txt`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./complex_commands
Complex Commands Program
Supported format:
cat < input.txt | grep WORD > output.txt

Enter command: cat < missing_q15.txt | grep OS > output1.txt
Input file: No such file or directory
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

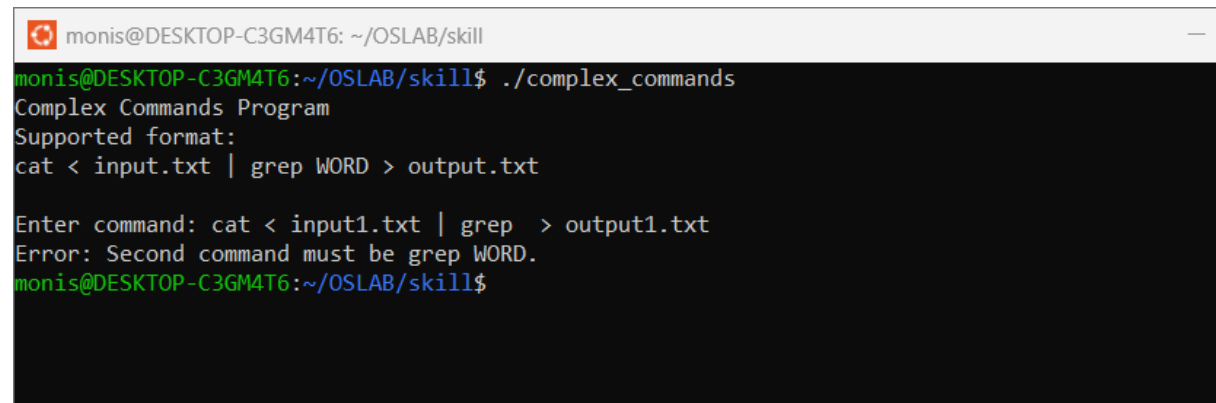
Test C — Missing search word

Run:

`./complex_commands`

Enter:

`cat < input1.txt | grep > output1.txt`

A terminal window with a title bar showing 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill'. The terminal content shows the command './complex_commands' being executed. The program outputs 'Complex Commands Program' and 'Supported format: cat < input.txt | grep WORD > output.txt'. It then prompts 'Enter command:' and receives 'cat < input1.txt | grep > output1.txt'. An error message follows: 'Error: Second command must be grep WORD.' The prompt returns to 'monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$'.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./complex_commands
Complex Commands Program
Supported format:
cat < input.txt | grep WORD > output.txt

Enter command: cat < input1.txt | grep > output1.txt
Error: Second command must be grep WORD.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 8 — Final verification

After completing the successful test and all three error tests, run:

`cd ~/OSLAB/skill`

**`ls -l combined_redirection.c combined_redirection complex_commands.c
complex_commands`**

Verify the successful pipeline output once more:

`cat output1.txt`

Check that it contains exactly two lines:

`wc -l output1.txt`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l combined_redirection.c combined_redirection complex_commands.c
-rwxr-xr-x 1 monis monis 16696 Sep 25 10:58 combined_redirection
-rw-r--r-- 1 monis monis 3190 Sep 25 10:57 combined_redirection.c
-rwxr-xr-x 1 monis monis 16944 Sep 25 11:53 complex_commands
-rw-r--r-- 1 monis monis 6003 Sep 25 11:45 complex_commands.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat output1.txt
OSSP Practical Q15
OS Process Management
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ wc -l output1.txt
2 output1.txt
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Open Ubuntu and run:

cd ~/OSLAB/skill

Check your programs:

**ls -l combined_redirection.c combined_redirection complex_commands.c
complex_commands**

Verify the combined output:

cat combined.txt

Verify the pipeline output:

cat output1.txt

wc -l output1.txt

```
Select monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l combined_redirection.c combined_redirection complex_commands.
c complex_commands
-rwxr-xr-x 1 monis monis 16696 Sep 25 10:58 combined_redirection
-rw-r--r-- 1 monis monis 3190 Sep 25 10:57 combined_redirection.c
-rwxr-xr-x 1 monis monis 16944 Sep 25 11:53 complex_commands
-rw-r--r-- 1 monis monis 6003 Sep 25 11:45 complex_commands.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat combined.txt
Command output: test
Error output: simulated error message
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat output1.txt
OSSP Practical Q15
OS Process Management
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ wc -l output1.txt
2 output1.txt
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 9 — Test Additional Scenarios

Test A — No matching data

Run:

./complex_commands

Enter:

cat < input1.txt | grep XYZ123 > no_match.txt

Verify that the output file is empty:

wc -l no_match.txt

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./complex_commands
Complex Commands Program
Supported format:
cat < input.txt | grep WORD > output.txt

Enter command: cat < input1.txt | grep XYZ123 > no_match.txt

Execution Plan:
1. Open input file: input1.txt
2. Execute cat
3. Send cat output through a pipe
4. Execute grep with search word: XYZ123
5. Redirect grep output to: no_match.txt
Pipeline completed. No matching lines found.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Test B — Verify execution order and output

Run:

`./complex_commands`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./complex_commands
Complex Commands Program
Supported format:
cat < input.txt | grep WORD > output.txt

Enter command: cat < input1.txt | grep OS > final_q15.txt

Execution Plan:
1. Open input file: input1.txt
2. Execute cat
3. Send cat output through a pipe
4. Execute grep with search word: OS
5. Redirect grep output to: final_q15.txt

Pipeline executed successfully.
Output saved to: final_q15.txt
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

